

59. La caméra : c'est le monde qui bouge

Le héros ne se déplace pas — comme dans Mario.

1. Ce qu'il faut comprendre

Dans un jeu qui défile, le personnage **ne bouge pas à l'écran**. Il reste au milieu, et c'est le décor qui passe derrière lui. Mario n'avance pas : il piétine au centre pendant qu'un monde de plusieurs écrans glisse vers la gauche.

Il y a donc **deux espaces** : l'écran, qui fait 160 pixels de large, et le monde, qui fait ce qu'on veut.

`defiler(camera, 0)` dit simplement **quel morceau du monde on regarde**.

Et voici pourquoi cela tient en si peu de lignes : **les lutins ne défilent pas**. `defiler()` ne déplace que le fond ; un `sprite()` est posé sur l'écran, à la place où on l'écrit. Le héros au milieu, c'est une ligne, toujours la même :

```
sprite(0, 76, 112, HEROS);
```

Une seule variable suffit pour les deux rôles. `camera` est la fenêtre **et** la position du héros dans le monde : il est en `camera + 76`. Le jour où un ennemi arrive, lui vit dans le monde, et c'est à lui de se convertir — `sprite(1, xEnnemi - camera, 112, ENNEMI)`.

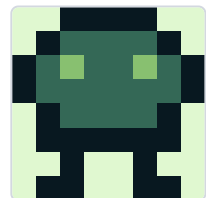
La preuve est à l'écran, et elle est gratuite : **même la consigne s'en va**. Elle est écrite dans le décor, donc elle voyage avec lui. Le bonhomme, lui, ne bouge pas d'un pixel.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1673 octets** de cartouche.

1. UN DESSIN — HEROS

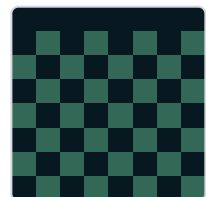
```
Tuile HEROS = {  
  "00333300",  
  "03222230",  
  "32122123",  
  "32222223",  
  "03222230",  
  "03333330",  
  "00300300",  
  "03300330",  
};
```



à l'écran

2. UN DESSIN — SOL

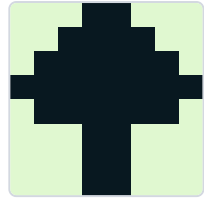
```
Tuile SOL = {  
  "33333333",  
  "32323232",  
  "23232323",  
  "32323232",  
  "23232323",  
  "32323232",  
  "23232323",  
  "32323232",  
};
```



à l'écran

3. UN DESSIN — ARBRE

```
Tuile ARBRE = {  
    "00033000",  
    "00333300",  
    "03333330",  
    "33333333",  
    "03333330",  
    "00033000",  
    "00033000",  
    "00033000",  
};
```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

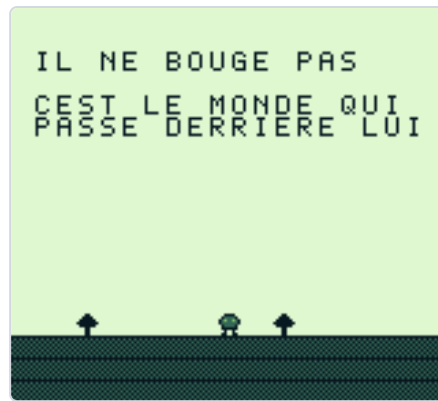
```
int main() {  
    uint8_t camera = 0;  
  
    texte(1, 2, "IL NE BOUGE PAS");  
    texte(1, 4, "CEST LE MONDE QUI");  
    texte(1, 5, "PASSE DERRIERE LUI");  
  
    for (uint8_t c = 0; c < 32; c++) {  
        poser(c, 15, SOL);  
        poser(c, 16, SOL);  
        poser(c, 17, SOL);  
    }  
  
    poser(3, 14, ARBRE);  
    poser(12, 14, ARBRE);  
    poser(25, 14, ARBRE);  
  
    while (true) {  
        image();  
  
        if (bouton(DROITE)) camera = camera + 1;  
        if (bouton(GAUCHE)) camera = camera - 1;  
  
        defiler(camera, 0);  
        sprite(0, 76, 112, HEROS);  
    }  
  
    return 0;  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



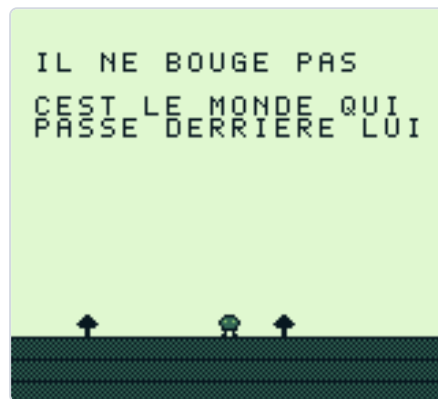
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Le bonhomme reste planté au milieu ; les arbres, le sol et même la consigne passent derrière lui.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le héros est posé au milieu de l'écran (76, 112)
- ✓ le monde est plus large que l'écran
- ✓ des arbres le jalonnent
- ✓ DROITE fait glisser le monde (0 vers 40)
- ✓ GAUCHE le ramène (40 vers 20)
- ✓ et le héros n'a pas bougé d'un pixel (76, 112)